

Data Mining 2 Project

Camilla Maffeis

Giulia Fabiani

A.Y. 2024/2025

CONTENTS

1	Data Understanding and Preparation	2
1.1	Data Semantics, Duplicates and Train/Test Split	2
1.2	First Global Exploration of the Training Set	2
1.3	Distribution of Categorical Variables and Statistics	2
1.4	Distribution of Numeric Variables and Statistics	3
1.5	Variable Transformation, Pairwise Correlations and Elimination of Variables	3
1.6	Handling Missing Values	6
1.7	Clustering	6
2	Advanced Data Pre-Processing	6
2.1	Anomaly detection	6
2.2	Imbalanced Learning	6
2.2.1	Resampling techniques	6
2.2.2	Decision threshold and assessment	8
2.2.3	Exploration of alternative techniques	9
3	Advanced Classification	9
3.1	Pre-processing	9
3.2	Logistic Regression	9
3.2.1	Rating	9
3.2.2	Title Type	10
3.3	Linear Support Vector Machine	10
3.3.1	Rating	10
3.3.2	Title Type	10
3.4	Support Vector Machine with Radial Basis Function (RBF)	11
3.4.1	Rating	11
3.4.2	Title Type	11
3.5	Random Forest	11
3.5.1	Rating	12
3.5.2	Title Type	12
3.6	LightGBM	13
3.7	Multilayer Perceptron	13
3.8	Conclusions	13
4	Advanced Regression	13
4.1	Random Forest Regressor	13
4.2	XGBoost Regressor	13
5	Time Series Analysis	13
5.1	Data Understanding and Preparation	13
5.1.1	Univariate analysis of variables	14
5.1.2	Bivariate analysis of variables	14
5.1.3	Visualization	14

1 DATA UNDERSTANDING AND PREPARATION

The goal of this project is to analyze two datasets providing different kinds of information about movies, TV shows, and other forms of visual entertainment.

In the following section, we'll focus on the tabular dataset, while the overview about the time series dataset composition and preparation can be found in Section 5.

This dataset is an extension of the one proposed in DM1: the records include key information about titles, insights into critical aspects such as awards and reviews, as well as ratings and further data about the title's production.

1.1 Data Semantics, Duplicates and Train/Test Split

The tabular dataset contains 149,531 records and 32 attributes; the nine attributes that were added to the original set from the DM1 dataset are reported in Tab. 1. This makes a total of nine categorical attributes, three binary attributes and 20 numerical attributes. Among the latter, only *runtimeMinutes* and *averageRating* can be classified as continuous, as the others mostly represent counts.

A first data quality check revealed only ten genuine duplicates, i.e. records sharing all feature values, which we dropped.

We then split the dataset, reserving 70% for training, and the remaining 30% for testing. As to avoid data leakage, we performed all subsequent data exploration exclusively on the training set. At the end of the data preparation stage, we applied all transformation and cleaning steps to the test set as well.

1.2 First Global Exploration of the Training Set

A first global analysis of the dataframe's summary and descriptive statistics revealed missing values for *endYear* (100,729), *runtimeMinutes* (28,185), *countryOfOrigin* (27,961) and *genres* (1,887). Since most records are missing an *endYear*, and the feature is strongly correlated to *startYear*, we dropped it.

Since a great majority of records had no information about sound mixes, we decided to drop this feature as well.

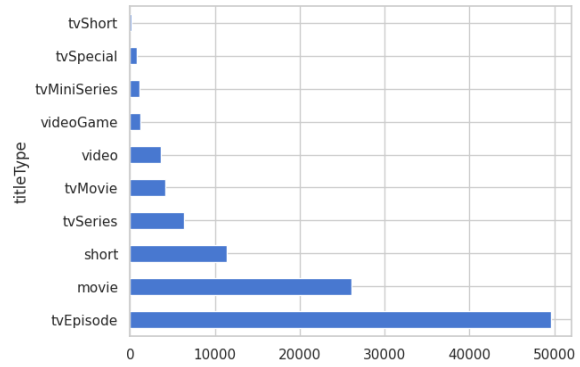


Figure 1: Distribution of *titleTypes*.

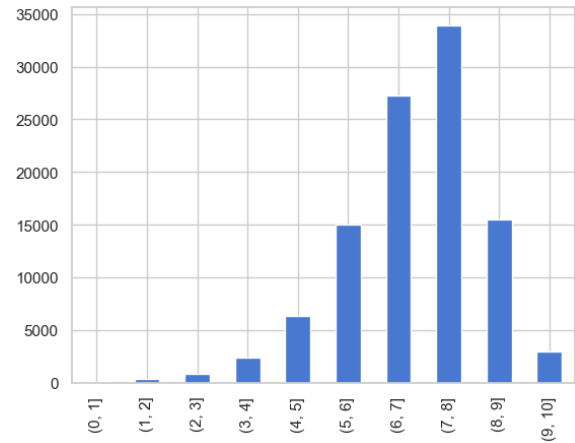


Figure 2: Barplot for *rating*.

bestRating, *worstRating*, and *isRatable* were once again confirmed to be uninformative, as they only take one value for all records (10, 1, and 1, respectively), and were therefore dropped. We also dropped *ratingCount*, as it was strongly correlated to *numVotes*, and *isAdult*, because it's redundant w.r.t. the *adult* category in *genres*.

Checking the values for the only remaining original binary attribute, *canHaveEpisodes*, we noticed that only two records with a *titleType* different from *tvSeries* and *tvMiniSeries* have a positive value. We corrected them: the first had been registered as a *tvMovie* instead of *tvSeries*, while the second was a *short* with no episodes.

Lastly, we removed the string '\\N', which was prepended to all lists of *regions*, making the feature consistent with *numRegions*.

1.3 Distribution of Categorical Variables and Statistics

In this subsection, we studied the univariate and bivariate distribution of categorical variables, which all turned out to be unbalanced.

Feature	Description	Type
castNumber	Total Number of Cast individuals present within the IMDb title page.	Discrete. Ratio-scaled.
companiesNumber	Total Number of companies that worked for the title.	Discrete. Ratio-scaled.
regions	The regions for this version of the title. Some titles can belong to more than one class.	Categorical. 171 unique classes.
averageRating	Weighted average of all the individual user ratings.	Continuous. Ratio-scaled.
externalLinks	Total Number of External Links the title has within the IMDb page.	Discrete. Ratio-scaled.
quotesTotal	Total Number of quotes the title has within the IMDb page.	Discrete. Ratio-scaled.
writerCredits	Total number of writer credits of the title.	Discrete. Ratio-scaled.
directorCredits	Total number of director credits of the title.	Discrete. Ratio-scaled.
soundMixes	Technical specification of the sound mixes available for the title. Some titles can belong to more than one class.	Categorical. 59 unique classes.

Table 1: New features in the extended dataset.

Differently from the DM1 dataset, the most popular *titleType* is *tvEpisode* (47% of the records), followed by *movie* (25%) and *short* (11%); *tvSpecial* (0.79%) and *tvShort* (0.17%) are the classes with the lowest support (cf Fig 1). With regard to *genres*, the two most popular classes are by far *Drama* and *Comedy*, although it depends on *titleType*. Binned *rating* follows a left-skewed distribution, with the mode being the [7,8) class (Fig. 2). The most popular *countryOfOrigin* and *region* is the US, but both features have a substantial amount of missing values: *nan* is the second most popular value for both.

We dropped *soundMixes*: not only are most of its 59 unique values very rare, it has too many missing values to provide useful information.

1.4 Distribution of Numeric Variables and Statistics

Before analyzing numeric variables, we added the same features we had added to the DM1 dataset:

- *numCountriesOfOrigin*: the number of countries listed for the record for the variable *countryOfOrigin*. For the training set, the values range from 1 to 10.
- *numGenres*: the number of genres listed in *genres*. For the training set, the values range from 1 to 3.
- *reviewsTotal*: the sum of *criticReviewsTotal* and *userReviewsTotal*.
- *criticReviewsRatio*: the proportion of review from critics (*criticsReviewTotal*) on the total number of reviews of a record (*reviewsTotal*). For unreviewed records, it is set to zero.
- *awardsAndNominations*: the sum of *awardWins* and *awardNominationsExcludeWins*.

A pairplot of all numeric variables, with KDE graphs on the diagonal, allowed us to gain a

global view of both the features' univariate distribution and their pairwise correlations. Given the size of the dataset, we resorted to using a random sample of 20,000 records. The great majority of the variables has a right-skewed distribution (*writerCredits*, *numRegions*, *awardWins*, *totalImages*, *criticReviewsTotal*, *runtimeMinutes*, *userReviewsTotal*, *totalCredits*, *totalVideos*, *castNumber*, *numVotes*, *quotesTotal*, *awardNominationsExcludeWins*, *companiesNumber*, *externalLinks*, *awardsAndNominations*, *numCountriesOfOrigin*, *reviewsTotal*). *averageRating*, of course, follows the same left-skewed distribution observed for its discretized version, *rating* (cf. Fig. 2).

We observed that some records have an implausibly long runtime, probably due to the inconsistent strategy adopted for the computation of the runtime for serialized content, for which sometimes the total runtime of a series is registered, rather than the average episode runtime. In order to avoid this problem, we removed the runtime of all *tvSeries*, *tvMiniSeries*, and *tvEpisodes* for which the value exceeded the 3rd quartile + 1.5 interquartile range for its specific category (105, 382.5 and 84 minutes, respectively); a total of 1560 records in the training set were affected.

A similar problem was found for *writerCredits*: records with suspiciously high values are likely TV episodes for which all writers of the entire TV series were erroneously credited. Therefore, we decided to remove the values for *writerCredits* from 2100 *tvEpisodes*, as they exceeded the 3rd quartile + 1.5 interquartile range (8.5 credits).

1.5 Variable Transformation, Pairwise Correlations and Elimination of Variables

We log-transformed all right-skewed variables, and used the Shapiro-Wilk test to determine whether they now resembled a normal distribution. None of them did, therefore we used Spear-

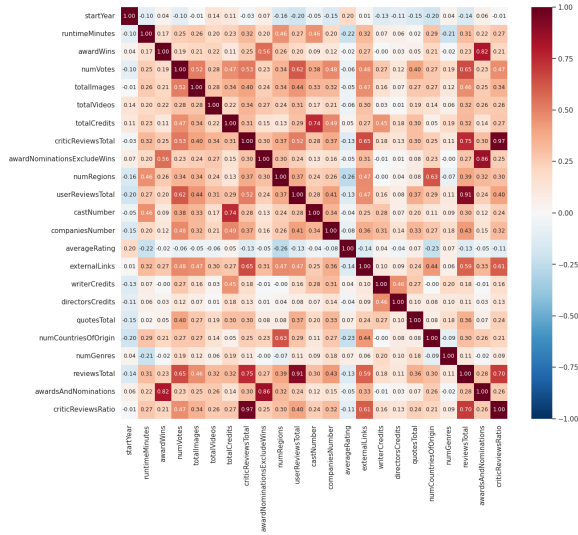


Figure 3: Correlation heatmap before attribute removal.

Original feature	Boolean version	% True
awardWins	hasAwards	8.16
awardsAndNominations	hasAwardsOrNominations	12.25
totalVideos	hasVideos	7.93
criticReviewsTotal	hasCriticsReviews	24.88
userReviewsTotal	hasUserReviews	36.98
directorsCredits	moreDirectorsCredits	9.89
quotesTotal	hasQuotes	13.86
numCountriesOfOrigin	moreCountriesOfOrigin	5.05

Table 2: Binarization of numeric features.

man’s correlation coefficient instead of Pearson in plotting the correlation heatmap in Fig 3.

We removed the following features because they were highly correlated to another feature: *awardNominationsExcludeWins* (0.86, correlated with *awardsAndNominations*), *castNumber* (0.74, with *totalCredits*). Since all features regarding reviews and *numVotes* were highly correlated with each other, we dropped them all with the exception of *numVotes*, which has the least skewed distribution, and *criticReviewsRatio*, which offer insights of a different nature. We however decided to conserve information about the presence/absence of reviews by users and critics with the creation of two new boolean features: *hasUserReviews* and *hasCriticsReviews*.

Moreover, we binarized features which overwhelmingly took on the value 0 (or 1, in the case of *numCountriesOfOrigin* and *directorsCredits*). The results of these transformations are reported in Table 2.

The total number of final features is 27; they are listed in Tables 3 and 4. Among the remaining numeric features, the heatmap in Fig. 4 shows that *externalLinks* exhibits a moderate to high positive correlation with *criticReviewsRatio* (0.61) and a low to moderate correlation with *numRegions*, *numVotes* and *totalImages* (around 0.47), while *numVotes* is also mildly correlated to *totalImages*, *totalCredits*, *criticReviewsRatio*, and *companiesNumber* (between 0.47 and 0.52).

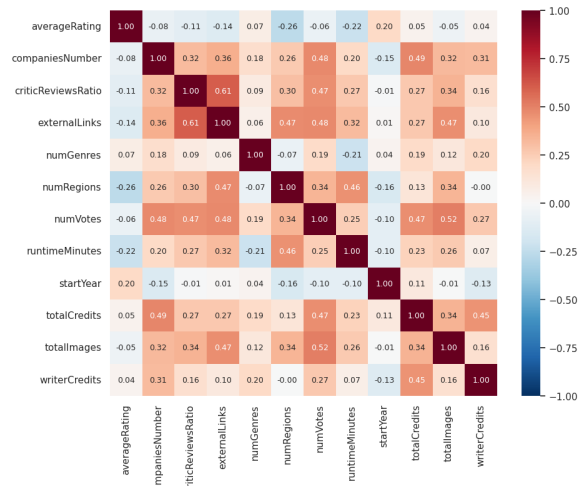


Figure 4: Correlation heatmap after attribute removal.

Feature	Description	Type
runtimeMinutes	Primary runtime of the title, in minutes.	Continuous. Ratio-scaled.
startYear	Represents the release year of a title. In the case of TV Series, it is the series' start year.	Discrete. Interval.
numVotes	Number of votes the title has received.	Discrete. Ratio-scaled. Log-transformed.
numRegions	The regions number for this version of the title.	Discrete. Ratio-scaled. Log-transformed.
totalImages	Total number of images for the title within the IMDb title page.	Discrete. Ratio-scaled. Log-transformed.
totalCredits	Total number of credits for the title.	Discrete. Ratio-scaled. Log-transformed.
criticReviewsRatio	The proportion of reviews from critics on the total number of reviews of a record. For unreviewed records, it is set to 0.	Continuous. Ratio-scaled.
numGenres	The number of genres listed for the variable <i>genres</i> .	Discrete. Ratio-scaled.
companiesNumber	Total number of companies that worked for the title.	Discrete. Ratio-scaled.
externalLinks	Total number of external links the title has within the IMDb page.	Discrete. Ratio-scaled.
writerCredits	Total number of writer credits of the title.	Discrete. Ratio-scaled.
averageRating	Weighted average of all the individual user ratings.	Continuous. Ratio-scaled.

Table 3: Numeric attributes after data preparation.

Feature	Description	Type
originalTitle	Original title, in the original language.	Categorical; mostly unique classes (i.e. names).
titleType	The type/format of the title (e.g. movie, short, tvseries, tvepisode, video, etc.).	Categorical, 10 classes.
countryOfOrigin	The country where the title was primarily produced.	Categorical, 153 classes. Some titles can belong to more than 1 class.
genres	The genre(s) associated with the title (e.g., drama, comedy, action).	Categorical, 29 classes. Some titles can belong to more than 1 class.
regions	The regions for this version of the title.	Categorical, 171 unique classes. Some titles can belong to more than 1 class.
rating	IMDb title rating class.	Ordinal, 10 values.
hasAwards	Whether the title has won at least an award	Asymmetric binary.
hasAwardsOrNominations	Whether the title was at least nominated for an award.	Asymmetric binary.
canHaveEpisodes	Whether the title can have episodes.	Asymmetric binary.
hasVideos	Whether the title IMDb title page has at least one video.	Asymmetric binary.
moreCountriesOfOrigin	Whether the title was produced in more than one country.	Asymmetric binary.
hasCriticsReviews	Whether the title has any critic reviews.	Asymmetric binary.
hasUserReviews	Whether the title has any user reviews.	Asymmetric binary.
moreDirectorsCredits	Whether the title has more than one director credits.	Asymmetric binary.
hasQuotes	Whether the title has any quotes within the IMDb page.	Asymmetric binary.

Table 4: Categorical, ordinal, and binary attributes after data preparation.

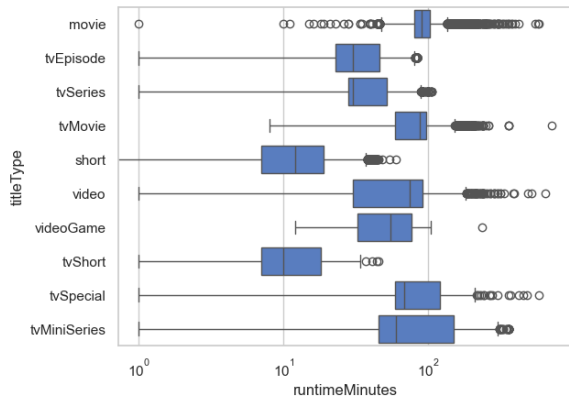


Figure 5: Distribution of runtimes across *titleTypes*.

1.6 Handling Missing Values

At this point, we still have missing values for *countryOfOrigin*, *region*, *genres*, *runtimeMinutes* and *writerCredits*; some were preexisting, others were removed as detailed Subsec. 1.4. Since the distribution of *runtimeMinutes* depends on *titleType* (cf. Fig 5), and the same holds true for *writerCredits*, we imputed their missing values using the median value within each respective class.

On the other hand, we chose not to replace missing values for *genres*, *region* or *countryOfOrigin*. They can easily be handled when performing the encoding for our methods without introducing bias, therefore we'd rather preserve the original distributions and correlations.

1.7 Clustering

As part of the data understanding phase, we conducted hierarchical clustering on the dataset, using all numeric attributes.

Since the size of the dataset made it unfeasible to perform hierarchical clustering directly because of the computational cost of computing distances between all records, we firstly reduced our dataset to a more manageable 10,000 records, running k-means clustering with $k=10,000$.

We then performed hierarchical clustering on the centroids of K-Means clusters, experimenting with different linkage methods and inspecting the resulting dendrograms. We found that using average linkage yielded the most satisfactory results. We then cut the dendrogram at an appropriate threshold and, after propagating cluster labels back to the entire dataset, analyzed the composition of the clusters.

We found 13 clusters of varying sizes, with the top cluster grouping one fifth of the entire dataset and the top four clusters grouping half of the

dataset. Five clusters contained less than 5,000 records each.

For reasons that will become apparent in 2.1, we concentrated our analysis on two smaller clusters: C1 (1,668 records) and C2 (3,923). These clusters belong to the same supercluster, that is merged latest of all with the rest of the dataset.

Both clusters are made up primarily from movies (with some tv series), and according to their distribution w.r.t. *numVotes*, probably contain blockbuster movies and very popular TV series 6. In addition to that, almost 57% of records from C1 received awards, a proportion that increases to 80% when considering also nominations. C2 records were also well received, with 45% of them having received or being considered for awards. In addition to that, 87% of records from C1 and 45% of records from C2 have videos on their IMDB page and almost all records from both C1 and C2 have both reviews from users and from critics.

2 ADVANCED DATA PRE-PROCESSING

2.1 Anomaly detection

2.2 Imbalanced Learning

In this section we will discuss several imbalance learning techniques in the context of binary classification. Our goal was to develop a system that is capable of predicting which titles have received any awards (as already seen in 2, only about 8% of our training records did). We can imagine that a similar system could be useful in a number of contexts, like for recommendation and showcasing purposes or for streaming platforms attempting to decide which titles to license when this information isn't already available (for example for newly released titles).

In our evaluation of different techniques, we decided to rely on the F1 score of the positive class, since we figured that from a business perspective it was equally important detecting as many award worthy titles as possible and avoiding false positives.

2.2.1 Resampling techniques

We established a performance baseline using a KNN classifier with no resampling techniques. We then experimented with three over-sampling strategies (Random Over-sampling,

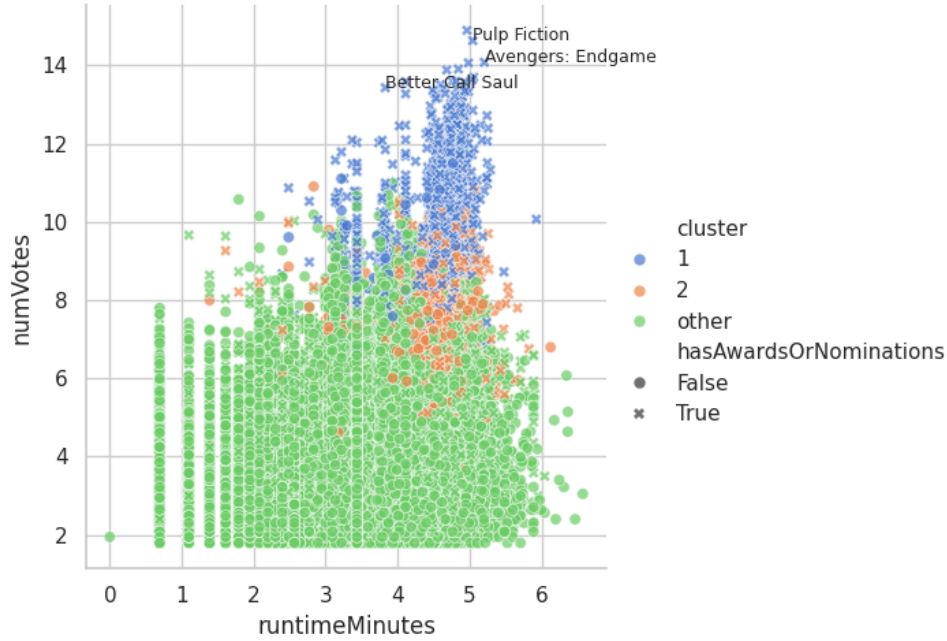


Figure 6: Hierarchical clustering results on tabular data by runtime and number of votes.

SMOTE, ADASYN), two under-sampling strategies (Random Under-sampling and Edited Nearest Neighbors) and a combination of both (SMOTEEN). In all cases we resampled the dataset as to obtain a perfect balance between the positive and the negative class.

For each technique we fine tuned the hyperparameters of the classifier searching for the best F1 score: initially we performed random search on k s between 1 and 100, with euclidean and manhattan distance, both weighting neighbors and not. Since we noticed generally better performance with weighting of exemplars, we removed unweighted models from the grid search.

We also noticed that for over-sampling methods performance was rapidly decreasing as k got bigger, so we restricted our grid search to odd k s between 1 and 21. Since performance of under-sampling techniques wasn't degrading so rapidly, we extended the number of neighbors considered for these approaches to odd numbers between 1 and 41, but we noticed that scores were plateauing around 10, so we didn't consider it necessary to widen the search beyond that.

We then compared performance of different resampling strategies checking precision, recall and F1 score on 5-fold cross validation (cf fig 5).

Precious insights were also gathered by the inspection of the precision-recall curves (reported in fig 7) and average precision.

Over-sampling methods and the combination of SMOTE and Edited NN all show worse rank-

Method	precision	recall	f1
No strategy	.47 $\pm$.02	.31 $\pm$.01	.37 $\pm$.01
Random Over-saml.	.29 $\pm$.00	.62 $\pm$.01	.40 $\pm$.01
SMOTE	.29 $\pm$.00	.65 $\pm$.01	.40 $\pm$.01
ADASYN	.30 $\pm$.00	.59 $\pm$.01	.40 $\pm$.00
Random Under-saml.	.24 $\pm$.00	.86 $\pm$.01	.37 $\pm$.00
Edited NN	.50 $\pm$.01	.45 $\pm$.01	.47 $\pm$.01
SMOTE + Edited NN	.30 $\pm$.01	.68 $\pm$.01	.41 $\pm$.01
One Class SVM	.10 $\pm$.00	.62 $\pm$.01	.17 $\pm$.00

Table 5: Results of tested resampling methods on 5-fold cross validation.

ing capabilities than no resampling strategy at all. In particular these methods demonstrate much worse performance in the early retrieval range of operating points, meaning that they have much more false positives in the top ranked instances. This is also reflected in the lower precision scores that are not balanced by the higher recall. In addition to that, over-sampling techniques, even when relatively inexpensive at fitting time (Random Over-sampling), are slower at prediction time (because of the necessity to compute distances with all records in the over-sampled dataset) and have considerably higher space requirements.

Performance obtained with Random Under-sampling has similar F1 score to no resampling strategy, but shows better ranking capabilities, as demonstrated both by AP score and inspection of the PR curve.

The best results are obtained with Edited Nearest Neighbor. The PR curve for ENN almost

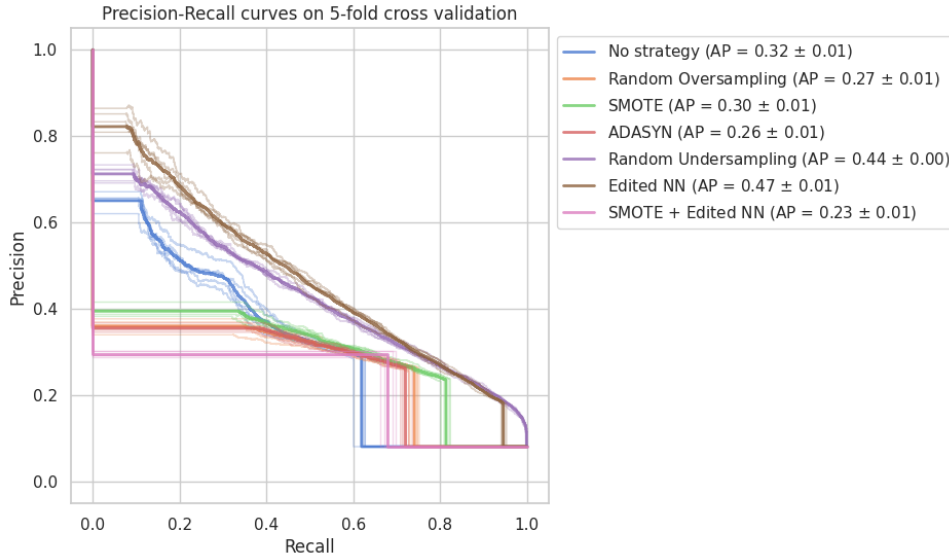


Figure 7: Precision-Recall curves on 5-fold cross-validation for different resampling methods

always dominates random under-sampling with minimal crossing-over for higher values of recall.

It seems to us that Random Under-sampling may be the preferred choice in some context in which fitting time would be a more pressing concern than the precision of the prediction, so we decided to assess both Edited NN and Random Under-sampling on the test set.

2.2.2 Decision threshold and assessment

Edited Nearest Neighbor showed satisfactory performance, improving by 0.1 the F1 score of the vanilla KNN and having the best precision among all tested methods (0.50) and recall at least better than the vanilla KNN. Nevertheless, we decided to post-tune the decision threshold of both our best performing models before assessment on the test set. For the pipeline using Random Under-sampling this resulted in setting a higher threshold (0.77), while for Edited NN the threshold had to be lowered to 0.38. Fig 8 illustrates the effect of changing the decision threshold for the pipeline using Random Under-sampling: we can observe that at the default threshold the model would have better recall but much worse precision than at the tuned threshold.

We finally assessed both models on the test set, obtaining very similar results: ENN obtained a F1 score of 0.49, while Random Under-sampling 0.47; inspecting the other metrics we can confirm that the worse performance of Random Under-sampling is entirely due to a lower precision (0.42 vs 0.44 for ENN), as we had expected from the inspection of the PR curves in validation.

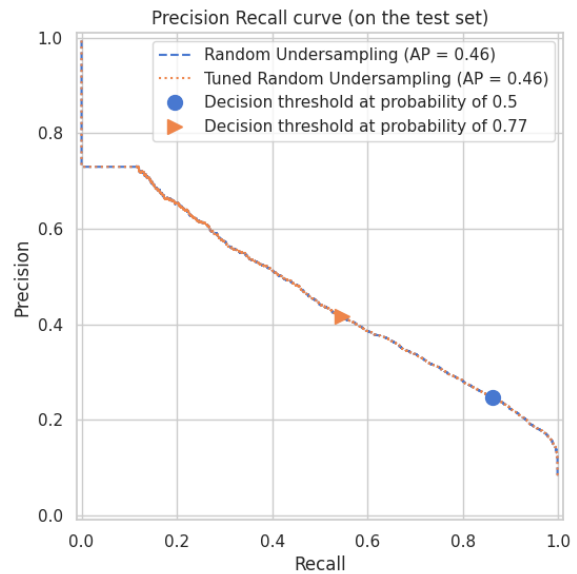


Figure 8: Decision threshold for KNN with Random Under-sampling before and after tuning.

2.2.3 Exploration of alternative techniques

As we detailed in 2.1, records that have received awards and nominations were often deemed anomalous. We decided to leverage this observation adopting a classification based anomaly detection technique for our classification task.

We trained a One Class SVM with RBF kernel without hyperparameter tuning on records that didn't receive any awards in order to learn a discriminative boundary around normal instances. Table 5 show results obtained by this method on 5-fold cross validation. We ultimately deemed the performance of this method unsatisfactory because it produced too many false positives.

3 ADVANCED CLASSIFICATION

In this section, we will examine the performance of various advanced classification methods w.r.t two multiclass classification tasks, aiming to predict the *rating* class and the *titleType* of our records.

Since no features showed high correlation with *averageRating*, while some did exhibit different distribution tendencies w.r.t. *titleType*, we anticipated that the second classification task would yield better results.

In evaluating and comparing our classification models, we selected macro F1 as the main performance metric, given that the different target classes have variable support. For each model, we additionally plotted the Precision-Recall (PR) curves for each class using the one-vs-rest strategy and reported its PR AUC. We favored PR curves instead of ROC curves because they are more informative in multiclass and imbalanced settings.

3.1 Pre-processing

For each model, we built a pipeline which proceeded to first preprocess the data (we used the training and test set, cleaned and prepared as detailed in Sec. 1) and then pass it to the model, in order for it to be fitted on the training set or applied on the test set.

The preprocessing step included scaling the numerical features, if required by the method, and one-hot encoding the categorical variables. For both tasks, we of course dropped the target variable, including *averageRating* for the rating task.

For *countryOfOrigin* and *region*, we only kept the 25 most frequent countries, aggregating the rest into an *Other* category, to reduce dimensionality. For the MultiLayerPerceptron Classifier (and, later, for the regression task), we further reduced dimensionality, keeping only the top nine countries for both features and dropping *hasAwards* on the account of its positive correlation to *hasAwardsOrNominations*, in order to further reduce computational costs and training time during hyperparameter tuning.

3.2 Logistic Regression

As a baseline, we first tried a Logistic Regression model. As the algorithm assumes that the features follow a Gaussian distribution, we scaled numerical features using PowerTransformer, which reduces skewness and stabilizes variance.

3.2.1 Rating

Using the default parameters yielded a macro F1 of 0.144 and an accuracy of 0.388. We performed a grid search using 5-fold cross validation, which resulted in a very small increase, with a 0.146 average macro F1 on the validation sets and 0.152 on the training set, which at least proves no overfitting. We tested the following hyperparameters (the values for the final model are in bold):

```
{ 'penalty': ['l2', 'l1', 'elasticnet'],
  'C': [0.001, 0.01, 0.1, 1, 10, 100],
  'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9],
  'solver': ['lbfgs', 'sag', 'saga', 'newton-cg',
             'newton-cholesky', 'liblinear'] }
```

On the test set, the final model achieved a **macro F1** of **0.145** and an **accuracy** of **0.388**, comparable to the default model. The poor performances across decision thresholds are confirmed by the **PR AUC** of only **0.193**.

The model has managed to generalize some information only for the classes with the highest support: the performances peak for the mode, (7,8] (F1: 0.544), its neighboring classes are confused for it most of the time, and as we move to the tail of the bell distribution, it achieves consistently poor results (F1 < 0.05).

Looking at the model's coefficients, the overall most important feature is *hasCriticReviews* (mean absolute value > 1.2, while for the other coefficients in the top 15 is in the interval [0.6, 0.7]): it is the first or second-biggest coefficient for the best performing classes (i.e. for the interval (5,8]), and for all of them it takes on a negative value, i.e. having critical reviews lowers the probability of a record of having an 'average' rating.

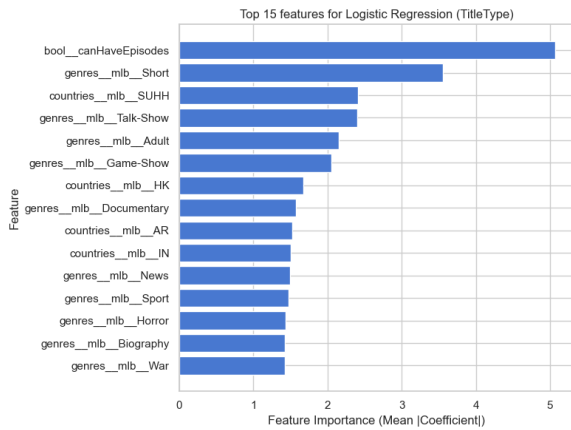


Figure 9: Top 15 most important features for *titleType* (LR).

3.2.2 Title Type

For this task, the final model resulting from the grid search achieved an average validation macro F1 of 0.653 (on the training set: 0.664). The final parameters chosen were: 'C': 100, 'penalty': 'l2', 'solver': 'newton-cg'.

On the test set, the model achieved a **macro F1** of **0.649** and an accuracy of **0.908**; the improvement w.r.t. the default parameters (0.647 and 0.908) is, again, quite small. The **PR AUC** is **0.704**; the curve for each class reflects its performance.

Given that both tasks have 10 target classes, our intuition about *titleType* yielding better results is confirmed.

The four most popular classes (*tvEpisode*, *short*, *movie*, *tvSeries*) achieve great results, with F1 > 0.9. The best performing and most popular class, *tvEpisode*, peaks with 0.978. All other classes (except *tvShort*) achieve a F1 score > 0.4, which is still significantly more than random chance. *Videogame*, in spite of its low support, manages to achieve very good results (F1 0.76).

The 15 most important features are reported in Fig. 9. Overall, the two most informative features are:

- *canHaveEpisodes*: it has very high positive values for *tvSeries* (+13.15) and *tvMiniSeries* (+12.18) and negative for all other classes; it's also the most important feature for *movie* (-4.75).
- *genre Short*: of course, it's the most important feature for *short* and *tvShort*, and it's the second most important for *video* with positive values; it's high up in the ranking with a negative value for *movie*, *tvEpisode*, *tvMovie*.

3.3 Linear Support Vector Machine

For Support Vector Machines (SVMs), we scaled numerical features using *RobustScaler*. We tested the linear kernel and the Radial Basis Function kernel separately to exploit *sklearn*'s optimized *LinearSVC* implementation.

3.3.1 Rating

For *LinearSVC*, we tested the following parameter grid using a grid search with 5-fold cross validation:

```
{ 'C': [0.001, 0.01, 0.1, 1, 10, 100],
  'penalty': ['l1', 'l2'],
  'loss': ['squared_hinge', 'hinge'],
  'max_iter': [5000],
  'class_weight': [None, 'balanced'] }
```

It resulted in an average macro F1 of 0.158 for validation and of 0.164 on the training set.

On the test set, w.r.t. Logistic Regression, the model achieved a higher **macro F1** (**0.158**), but a lower **accuracy** (**0.270**). Indeed, the F1 for the best performing classes has decreased ((7,8]: 0.496 vs 0.544; (6,7]: 0.215 vs 0.356, (5,6]: 0.259 vs 0.283), in favor of slightly better performances for classes with lower support: the decrease in accuracy is linked to the fact that the model tries to predict these lower support classes more often, rather than focusing merely on the central classes. This led to a notable increase in F1 for (3,4] (0.113 vs 0.04) and (9,10] (0.145 vs 0.046), but it is clear that the model still struggles. The **PR AUC** is also slightly worse, **0.175**.

The confusion matrix confirms this tendency: if for Logistic Regression predictions were concentrated on the best performing classes, with many zeros populating the lowest classes, now confusion is spread throughout.

The most important feature is *tvEpisode* (mean absolute value: 0.175): it is at number one, with negative values, for classes from (2,3] to (5,6] — this would suggest that serialized content is less likely to have bad rating. Classes (7,8] and (8,9] both have *movie* as most important feature, with negative contributions (-0.207 and -190, respectively). This might explain why the latter is mostly classified as the former, which is the class with the highest support.

3.3.2 Title Type

The model returned by the grid search yielded an average validation macro F1 of 0.646 (vs 0.654 on the training set); the parameters were 'C': 0.1, 'class_weight': 'balanced',

```
'loss': 'squared_hinge', 'max_iter':
5000, 'penalty': 'l1'.
```

On the test set, the results were slightly lower than Logistic Regression: **0.640 macro F1**, **0.889 accuracy**, **0.674 PR AUC**. Indeed, performances for almost all classes have decreased to some degree, with a remarkable dip in precision for *videogame* (0.486 vs 0.799).

The overall most important features are the same (*canHaveEpisodes* and genre *Short*), but now only the first stands out (mean absolute value close to 2.5), while *Short* has a similar value to the rest of the top features, < 1.

Unexpectedly, the most important feature for the *titleType short* is now *canHaveEpisodes*, with quite an 'advantage' (coef: -2.18, vs genre *Short* 1.43).

3.4 Support Vector Machine with Radial Basis Function (RBF)

The power of SVMs comes at a high computational cost, which is why we only tested one kernel function (RBF) and why we opted for an 80/20 holdout, instead of a k-fold cross-validation, to carry out hyperparameter tuning. We also defined a small hyperparameter grid:

```
{ 'C': [0.1, 1, 10],
  'gamma': ['scale', 0.01, 0.1],
  'class_weight': [None, 'balanced'] }
```

3.4.1 Rating

The best model returned by the grid search had the following hyperparameters: 'C': 10, 'gamma': 0.1, 'class_weight': None. It achieved a macro F1 of 0.265 on the validation set and of 0.698 on the training set, suggesting overfitting. We therefore reran the grid search, excluding the highest values for C and gamma in order to increase regularization. The best performing model was less overfitted, but also achieved lower results (macro F1 0.206 for validation and 0.316 for training). Its parameters are: 'C': 1, 'class_weight': 'balanced', 'gamma': 'scale'.

On the test set, the overfitted model proved to be the best one, achieving a **macro F1** of **0.257** (vs 0.219) and an **accuracy** of **0.431** (vs 0.314). The resulting **PR AUC** is **0.231**. This model is also the best performing one thus far, with improvements observed across almost all classes.

Looking at the confusion matrix, we can see that the model doesn't completely snub lower support classes, like Logistic Regression did, but

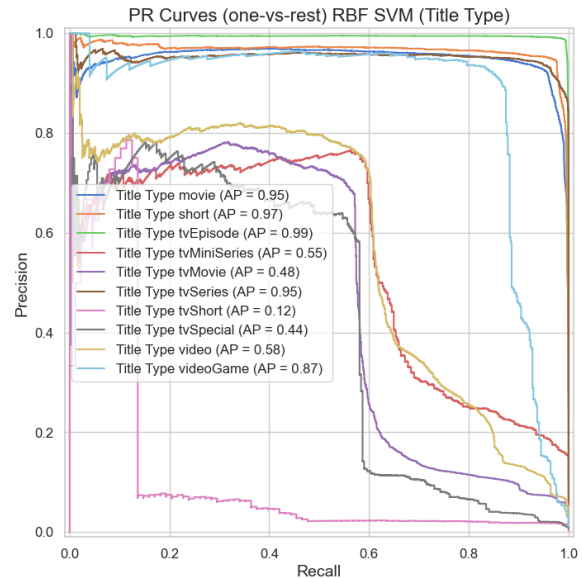


Figure 10: PR curves for *titleType* (SVM with RBF).

is more accurate than LinearSVC. Especially for the more popular (and better performing) classes, confusion is mostly among neighboring classes.

3.4.2 Title Type

Similarly to what had happened with *rating*, the first grid search returned an overfitted model (macro F1 for validation: 0.746, vs for training: 0.905). Forcing regularization resulted in less overfitting, but also in a much poorer performance (validation macro F1: 0.206, train macro F1: 0.316); for this reason, we chose the first classifier. Its parameters are: 'C': 10, 'class_weight': None, 'gamma': 'scale'.

On the test set, this model clears all previous attempts, with a **macro F1** of **0.746** and an **accuracy** of **0.938**.

The performances for all classes increased. *tvEpisode* now achieves an F1 of 0.99; *videogame* improved greatly, reaching 0.884 F1. Nevertheless, we only get a **PR AUC** of **0.689**; indeed, the curves in Fig. 10 show how the precision for half of the classes sinks at around 60% recall. This means that while the model shows excellent performance metrics on a per-class basis, it has difficulty maintaining a reliable performance when attempting to comprehensively identify instances of minority classes.

3.5 Random Forest

Since this method is based on trees, no scaling is needed for numerical features. To cut com-

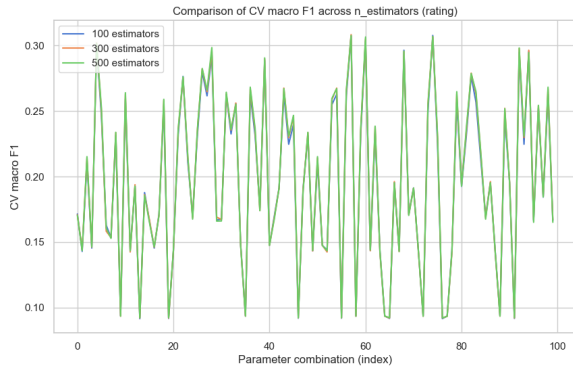


Figure 11: Avg cross-validation macro F1 for different parameter configurations across varying numbers of estimators for *rating* (RF).

putational costs, we used 3-fold cross validation during hyperparameter tuning.

Indeed, there are many possible hyperparameters to tune. For each task, we ran three Randomized Searches, for a varying number of estimators — 100, 300 and 500, respectively. For each, we tested the same 100 random combinations of parameters from the following grid:

```
{ 'criterion': ["gini", "entropy"],
  'max_depth': [None, 5, 15, 25],
  'min_samples_split': [2, 5, 10],
  'min_samples_leaf': [1, 5, 10],
  'class_weight': [None, 'balanced'] }
```

3.5.1 Rating

Before proceeding with hyperparameter tuning, we first tested a Random Forest using default parameters, which yielded an average validation macro F1 of 0.296, and an even better 0.302 macro F1 score on the test set.

As shown in Fig. 11, the number of estimators is not a strong driver of performance. Although the performance of the other two models was very close, we chose the best model with 300 base classifiers, which yielded an average macro F1 equal to 0.308 for validation, and equal to 0.968 for train. Its parameters are: `'n_estimators': 300`, `'min_samples_split': 5`, `'min_samples_leaf': 1`, `'max_depth': None`, `'criterion': 'gini'`, `'class_weight': 'balanced'`.

Whether such a gap between validation and train macro F1 should be seen as a sign of overfitting for Random Forest is debated in the literature: it has been argued that, even if the single trees can overfit, they tend to do so in different ways, and that combining them in an ensemble reduces the overall variance. Thus, although pruning the individual trees can be beneficial, unless the dataset

is very simple, we could consider Random Forest as particularly robust against overfitting.

The model's performance on the test set increases, making it the best model thus far: it achieves a **macro F1** of **0.325**, an **accuracy** of **0.452**, and a **PR AUC** of **0.318**.

It achieves the most balanced performance, with six classes out of ten scoring an $F1 \geq 0.3$. Confusion remains spread out for the minority classes (from 0 to 4), whereas for the others it remains concentrated amongst neighboring classes.

The Mean Decrease in Impurity (MDI) returned as the most important features *numVotes* and *startYear* (almost 0.10), followed by *totalCredits* (around 0.9).

3.5.2 Title Type

The default model achieved an average macro F1 of 0.707 on validation and a score of 0.728 on the test set.

The performances for the different parameters combinations tested during the Random Searches were again very similar, with the configurations using 100 estimators yielding marginally lower scores. Indeed, the three best models shared the same hyperparameter values. We chose the configuration with 300 trees: the validation performances were only marginally better than that of the 500-trees model, but we also favored simplicity.

The model yielded 0.767 and 0.975 for validation and training, respectively. Its hyperparameter configuration is the following: `'n_estimators': 300`, `'min_samples_split': 10`, `'min_samples_leaf': 1`, `'max_depth': None`, `'criterion': 'entropy'`, `'class_weight': 'balanced'`.

On the test set, it clears previous models, achieving **0.777 macro F1** and **0.940 accuracy**. Half of the classes yield $F1 > 0.95$. It generalizes better on the minority classes *tvMovie* and *tvMiniSeries*: although there is still confusion with the more popular *movie* and *tvSeries*, they are correctly classified most of the time (cf. the confusion matrix in Fig. 13).

This model's superiority across decision thresholds is supported by the PR curves in Fig. 12, and further quantified by its **PR AUC** of **0.818**.

Checking the 15 most important features (Fig 14), we see how the two most important features for the Logistic Regression model, *canHaveEpisodes* and the genre *Short*, have been overtaken by *runtimeMinutes*. Indeed, we had uncovered a relationship between *runtimeMinutes* and *titleType* in Subsec. 1.6.

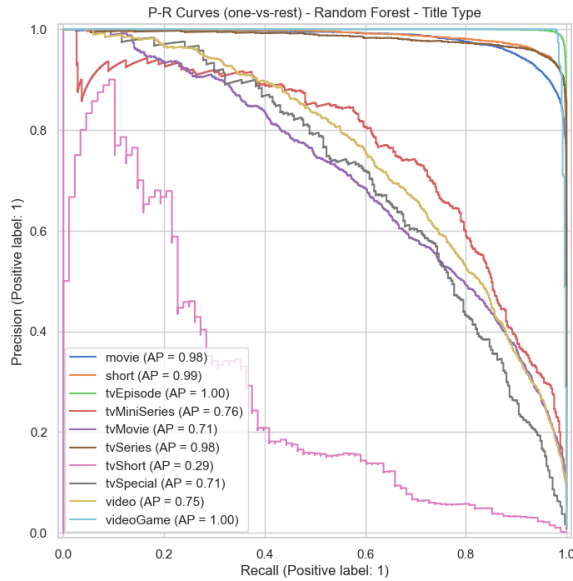


Figure 12: PR curves for *titleType* (RF).

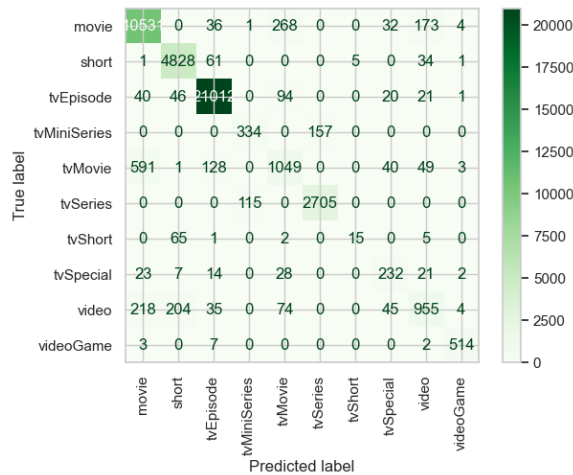


Figure 13: Confusion matrix for *titleType* (RF).

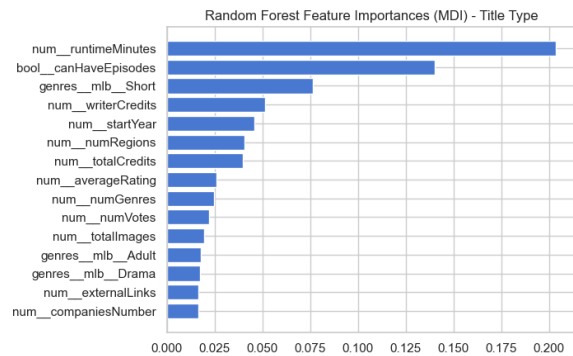


Figure 14: Top 15 most important features for *titleType* (RF).

Finally, it's worth pointing out that prior experimentation keeping the default hyperparameter 'class_weight': None instead of 'balanced' lead to a best model scoring a significantly lower macro F1 score of 0.728. This is indicative of the importance of this hyperparameter for this unbalanced task.

3.6 LightGBM

3.7 Multilayer Perceptron

3.8 Conclusions

4 ADVANCED REGRESSION

In this section, we will discuss the performance of two advanced regression methods in predicting the *averageRating*.

We used the same preprocessing pipeline detailed in Subsec. 3.1 and the feature set we had used for the Multilayer Perceptron.

We evaluated performances using the Determination Coefficient (R^2) and the Mean Squared Error (MSE). To set a baseline, we used a DummyRegressor which always predicts the mean of the target values observed in the training set, which yielded a R^2 of 0 and a MSE of 1.819 (equal to the variance of the target variable).

4.1 Random Forest Regressor

4.2 XGBoost Regressor

5 TIME SERIES ANALYSIS

5.1 Data Understanding and Preparation

This dataset contains data on the domestic (US and Canada) daily gross income at the box office for 1,134 movies, measured from their release date up to 100 days after release. The dataset was extracted from Box Office Mojo¹ and contains no missing values, because shorter time series were padded using noise augmented mean. Each time series was associated with the movie IMDB id, genre and rating (both continuous average rating and a binned version). In addition to the provided data, we used the IMDB free api² to retrieve some additional information about the titles represented in the dataset: title and release year of the movie, runtime (in minutes) number

¹ <https://www.boxofficemojo.com/>

² <https://imdbapi.dev/>

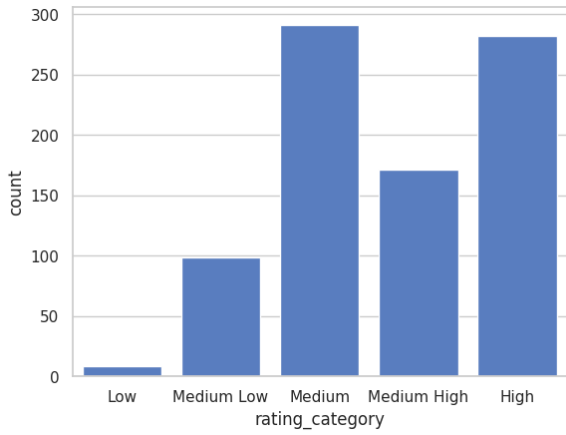


Figure 15: Distribution of rating category in the Time Series dataset.

of votes, countries of origin, number of awards and nominations. We additionally aggregated the box office data as a rough estimate of the total gross income of the movie.

We then splitted our initial dataset reserving 25% of records for testing and conducted the rest of our analysis exclusively on the training set, as to prevent data leakage.

5.1.1 Univariate analysis of variables

The distribution of number of votes, awards, nominations and total gross income show all a pronounced right skew, which is not unexpected. 90% of records had received awards or nominations. All records display normal runtimes for movies (between 90 and 200 minutes), except for 3 documentaries which were shorter than 45 minutes. We were able to detect some outliers also w.r.t. the release year: only two titles were released before 2010 (one in 1969 and one in 1984).

As for the distribution with respect to genres, any given movie can have more than one genre and the most popular genre was *Drama* (48%), followed by *Comedy* (36%), *Action* and *Adventure* (each of these 32%).

We also inspected the distribution of the binned version of rating, finding a considerable unbalance among classes, with *Medium* and *High* being the most frequent ones and the *Low* class representing less than 1% of titles (cf Fig. 15).

5.1.2 Bivariate analysis of variables

Figure 16 reports the Spearman correlation coefficient for pairs of numeric features. Besides obviously being correlated with each other, both awards and nominations are positively correlated with both the number of votes and rating. We also

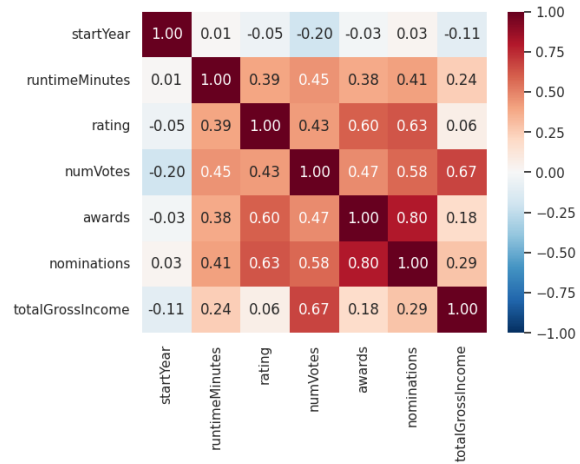


Figure 16: Correlation of numeric features.

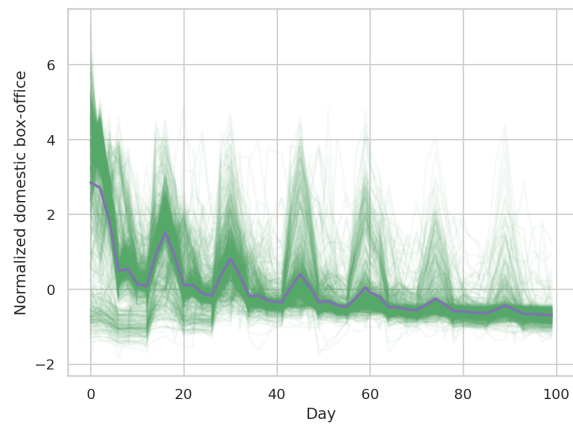


Figure 17: Time series (and mean time series, in purple) after amplitude scaling

observe a moderate to strong positive correlation between the estimated gross income and number of votes. Perhaps surprisingly, total gross income is only weakly correlated with rating.

Positive correlation of the number votes with rating is something we didn't observe in the IMDB dataset, were, on the contrary these two features were weakly negatively correlated. We think this may be due to the fact that movies have on average lower ratings than TV Episodes, which, in turn, have fewer votes than movies. Since the current dataset is limited to movies, we do not observe this (spurious) relationship anymore.

5.1.3 Visualization

We conclude our understanding visualizing the signals for all time series after amplitude scaling (fig. 17). Even from this simple visualization we are able to draw some interesting insights.

First of all our time series in general show slightly negative trend and some seasonality with

a cycle over around 14 timestamps: this observation was confirmed when we extracted Catch22 features from the time series and were able to verify that most of them had a periodicity of 13 or 14. This finding, which we initially took at face value, was somewhat puzzling, because we expected a day of week seasonality, with peaks in the weekends, while it made no sense for peaks to occur every other week. Surely enough, when we finally checked against the original data on Box Office Mojo, we were able to confirm that the observed peaks are, in fact, the effect of day of week seasonality, and that the time series were altered before we received them, either during the extraction or during the preprocessing.

In addition to this discovery, we also were able to observe that, while most movies have profits well over the mean in the first several days after release, some others got a slow start at the box office. We were able to identify 145 movies that performed below average in the first day of box office. While this is a very coarse distinction, which incurs in some false positives, it was later confirmed by clustering. In addition to lower box office profits in the first days after release, movies of this second type show no clear linear trend (in opposition to the negative trend observed for other movies) and tend to peak on the third weekend after release. We also observed that these were not, for the most part (93%), action movies.